

NEW UNIVERSE

GDD Addendum v1.1

— Δ-документ к Stellar_Forge_GDD.docx —

Что меняется:

- нейминг проекта • локальная разработка через Docker
- per-player генерация мира • неприкосновенная домашняя система
- межсистемные перелёты + спец. топливо • микросекторы общего пула
- лёд $\rightleftharpoons$ вода преобразование • per-player visibility

Этот документ дополняет основной GDD. Все разделы основного GDD сохраняют силу, если ниже не указано иное.

1. Нейминг проекта

Решение: основное имя продукта — New Universe. «Stellar Forge» был рабочим codename в первой версии GDD и остаётся в существующих файлах для трассируемости, но новые документы, репозитории и UI-элементы используют только New Universe.

Convention

Пользовательский нейминг (видимое игроком имя в Telegram, описание Mini App, странички продвижения): NEW UNIVERSE.

Кодовое имя (внутренний slack/git/issue tracker, может встречаться в commit-сообщениях): NU или NewUniverse.

Имя backend-сервиса: new-universe-api.

Имя frontend-сервиса: new-universe-tma (Telegram Mini App).

Имя БД: new_universe (snake_case для Postgres).

Имя Telegram-бота: @NewUniverseGameBot или @NU_Game_Bot (доступность проверить через @BotFather).

1.1. Что НЕ меняется

- Папка проекта на компьютере остаётся "New Universe" — она уже так названа.
- Существующие документы Stellar_Forge_GDD.docx, Stellar_Forge_Diagrams.html, Stellar_Forge_Infra_Costs.docx сохраняются как есть. В новых упоминаниях ссылаться на них с пометкой «(внутренний codename Stellar Forge)».
- Все игровые механики (ресурсы, корабли, бой, экономика) сохраняются.

1.2. Что меняется при формальной публикации

- Главная страница ТМА, описание в @BotFather, посадочная страница: всё строго New Universe.
- GitHub репозиторий: предлагается название new-universe (или new-universe-game).
- package.json/manifest: name = "new-universe-api" / "new-universe-tma".
- Docker images: ghcr.io/anry88/new-universe-api, ghcr.io/anry88/new-universe-tma.

2. Изменения в ресурсной системе

2.1. Лёд ⇄ Вода — двунаправленная конвертация

Дополнение к §3 GDD «Ресурсы». Базовая идея: эти ресурсы — фактически разные агрегатные состояния H_2O , но геймплейно различаются по применению (вода — жизнь/охлаждение, лёд — прыжковое топливо/охлаждение реакторов). Конвертация даёт игроку гибкость в случае избытка одного и нехватки другого.

Направление	Ресурс/здание	Стоимость энергии	Скорость	Потери
Вода → Лёд	Криогенный завод (новое здание lvl 1+)	5 Е на 100 H_2O	100 H_2O / 30 мин на lvl 1	5% (тает в процессе)
Лёд → Вода	Тепловая станция (или любой завод lvl 3+)	3 Е на 100 Льда	100 Льда / 15 мин	0%
Free на «океанической» планете	—	автоматическ и	—	океан = бесконечный буфер

2.2. Новое здание: Криогенный завод

- Категория: Производство.
- Зависимости: Космопорт lvl 2, Хим-завод lvl 3.
- Макс. уровень: 15. На каждом уровне +20% к скорости конвертации.
- Стоимость постройки lvl 1: 600 Сталь + 200 Си + 50 Mg.
- Время постройки lvl 1: 45 минут.

2.3. Криогенный завод как хранилище

Дополнительная функция: лёд можно хранить эффективнее воды (менее текуч). При активном Криогенном заводе склад вмещает +30% Льда сверх обычного capacity.

Это создаёт стратегию «эссенция»: на длинный отъезд / vacation-mode игрок может сконвертировать всю воду в лёд, чтобы не превысить сар склада.

2.4. Влияние на крафт

- Прыжковое топливо (см. §3.4) требует именно лёд, не воду — это даёт льду уникальный стратегический спрос.
- Boilerplate-эконом: при простом обмене на маркете 1 Лёд $\approx$ 1.2 H_2O (наценка за полезность).
- Композит-броня по-прежнему требует Mg, Ti, SiC — никаких изменений.

3. Новая модель генерации мира

Большое изменение к §4 GDD «Планеты, системы, галактика». Старая модель «10 000 пред-сгенерированных систем на сезон» заменяется на ленивую per-player генерацию + общий пул с микросекторами.

3.1. Общая идея

Модель v1.1

- 1) При создании игрока — генерируется ЕГО персональная домашняя система (Home System), неприкосновенная.
- 2) В домашней системе планеты не открыты заранее — игрок их находит, отправляя корабли в разные точки.
- 3) Чтобы выйти за пределы домашней системы — нужны: технология «Прыжковые двигатели», специальный Jump Ship, специальное топливо.
- 4) Прыжок переносит в Common Pool — общий мир.
- 5) Common Pool разбит на микросекторы. Системы внутри сектора генерируются лениво — когда нужен новый узел.
- 6) Видимость планет и систем — индивидуальная (per-player). Общая только базовая «карта секторов».

3.2. Home System — домашняя система игрока

- Создаётся при первом входе игрока. Сид = `hash(user_id, server_secret)`.
- Состав: 1 центральная звезда, 4-7 планет на разных расстояниях.
- Гарантии: стартовая планета (гарантированно содержит все 5 базовых ресурсов), 2-3 планеты с базой Tier 1, 1-2 планеты с одним Tier 2 ресурсом.
- Что отсутствует: Tier 3 (военные), Tier 4 (экзотика). За ними нужно лететь в общий пул.
- Минимум один источник «Третий» (новый Tier 3-минор) на 1 планете home system — это базовый компонент прыжкового топлива.
- Размер: радиус ≈ 100 ед. Планеты на дистанциях 5..80 от центра.
- Все планеты в home system — по умолчанию НЕ открыты. Открываются по visibility-правилам (см. §3.4).

3.3. Неприкосновенность Home System

- Никакой другой игрок физически не может попасть в Home System. Координаты не выводятся в общую карту, отсутствуют маршруты в неё.
- PvP в Home System невозможен. Ни нападения, ни шпионаж, ни блокада.
- Ресурсы Home System ограничены биомом и richness — вы НЕ можете построить там военный флот без специфических ресурсов из общего пула.
- Это создаёт чистое разделение: «безопасный экономический бэкэнд» = Home, «арена с риском» = Common Pool.

Дизайн-цель

Игрок никогда не теряет всё за ночь, пока спит. Базовая экономика защищена. Но интересный лут, военные ресурсы и прогрессия — только в Common Pool, где есть риск.

Это снимает основной страх idle/strategy игроков (грабёж в отсутствие) и одновременно сохраняет PvP-драйв.

3.4. Per-player visibility — открытие планет и систем

Все планеты и системы существуют в БД, но в UI игрока показываются только те, которые он «увидел». Видимость хранится отдельно для каждого игрока.

- Корабль имеет `sensor_range` (зависит от типа: Scout 30 ед, Recon Probe 60 ед, Cruiser 15 ед, Cargo 8 ед).
- При движении корабля сервер каждые N секунд (или при каждом тике) проверяет: есть ли невидимые для игрока планеты в радиусе `sensor_range`.
- Если есть — планета добавляется в `discovered_planets` таблицу для этого игрока, отправляется push-уведомление.
- То же для целых систем в Common Pool: игрок «открывает» систему, когда его корабль входит в её радиус (≈ 50 -100 ед).
- Технология Sensors прокачивает `sensor_range` на +50/+100/+200/+400/+800 ед. По tier 5 — Recon Probe видит 860 ед, что покрывает целые секторы за один полёт.

3.5. Межсистемные перелёты (Inter-system Jump)

Чтобы выйти за пределы Home System в Common Pool — нужны три вещи: технология, корабль, топливо.

Требование	Что именно	Как получить
Технология	Engines lvl 3 + новая ветка Jump Drive lvl 1	Лаб lvl 8 + 500 эл-ка + 200 Тритий
Корабль	Jump Ship (новый класс) — фрегат с Jump Drive	Военная верфь + 1500 Сталь + 500 Ti + 100 Тритий
Топливо	Прыжковое топливо (Jump Fuel)	100 Лёд + 50 Тритий + 1 ед. катализатор

3.6. Прыжковое топливо

- Базовый рецепт: 100 Лёд + 50 Тритий + 1 катализатор → 1 ед прыжкового топлива.
- 1 ед прыжкового топлива = 1 межсистемный прыжок (туда либо обратно).

- Расширенный рецепт (требует Антиматерию из Common Pool): 1 Антиматерия + 50 Лёд → 5 ед прыжкового топлива (×10 эффективность).
- Прыжковое топливо нельзя выстрелить из обычного двигателя — у Jump Ship отдельный бак.
- Тритий доступен только на 1 планете в Home System (низкая richness, регенерирует медленно). Это естественный «throttle» на скорость экспансии.

3.7. Common Pool — общий мир и микросекторы

Common Pool — это бесконечная сетка систем, разбитая на сектора. В отличие от Home System, сюда могут попасть все игроки.

- Размер сектора: $500 \times 500 \times 500$ ед (3D-куб).
- У каждого сектора есть seed (координаты сектора + server-secret), от которого зависит «потенциал» — массив возможных систем.
- В каждом секторе — желаемая плотность систем 3-8. Это фиксированный target, к которому стремится сервер.
- Системы внутри сектора имеют координаты внутри куба. Расстояние между ними — обычно 100-300 ед.
- Прыжок не привязан к точным координатам — игрок выбирает «направление и сектор», результат — попадание в одну из систем сектора (или в пустую точку, если сектор полупустой).

3.8. Когда генерировать новые системы

Lazy generation. Триггеры на серверной стороне:

1. При первом прыжке игрока в сектор: если в секторе < 3 систем, сервер генерирует ещё 2-4 систем рандомно.
2. При периодическом «density check» (раз в час): если активных игроков в секторе > capacity_threshold, добавить 1-2 системы.
3. При исследовательском прыжке: если игрок прыгает «в неизведанное», сервер всегда генерирует одну новую систему как «награду» за прыжок (с шансом 30% — тип «аномалия»).
4. Лимит: не более 12 систем на сектор (защита от инфляции мира).

3.9. Что не нужно генерировать заранее

- Систему common pool — генерится при первом обращении.
- Планеты внутри common-системы — генерятся при «открытии» системы (когда корабль входит в её радиус).
- Богатства (richness) каждой планеты — детерминированно от seed, считаются on-demand.
- Это даёт O(player) хранения и масштабирование на 100K+ игроков без дикой БД.

3.10. Бои в Common Pool

- Все PvP, рейды, грабежи происходят ТОЛЬКО в Common Pool системах (включая колонии игроков, которые там).
- Колонизация: игрок может колонизировать любую common-планету, отправив Колонизатор. После колонизации она становится его собственной, но НЕ становится частью Home System — атаковать её можно.
- NAP (защита новичков 7 дней) применяется к колониям в common pool, не к Home System (та неприкосновенна по умолчанию).

4. UI-карты: Home System, Sector Map, Common Pool

4.1. Home System Map

- 2D top-down вид собственной звёздной системы.
- Звезда в центре, планеты на орбитах, отрисованы только открытые. Неоткрытые видны как «?» в туманной зоне (если рядом проходил корабль).
- Корабли отрисовываются как треугольники с траекторией и ETA-таймером.
- Зелёный фон / синий — успокаивающий, чтобы было ощущение «дом».

4.2. Sector Map (Common Pool)

- Сетка 3D, отрисованная как 2D top-down с индикатором z-уровня.
- Видимы только сектора, в которых игрок имел корабль. Неисследованные — тёмные клетки.
- Внутри сектора — маркеры известных систем + точек интереса (анмалии).
- Цветовая палитра — холодная, индустриальная (серый/синий/оранж).

4.3. System Map (внутри common system)

- Похож на Home System Map, но с возможностью увидеть других игроков (если они колонизировали планету).
- Иконки рядом с планетой: владелец, флот в орбите, статус NAP.
- Кнопка «Запустить разведчика» / «Атаковать» / «Отправить колонизатор».

4.4. Mini-карта (overlay)

- В углу экрана: маленький overview, показывающий «ты сейчас здесь» (Home / Sector X-Y-Z / System X-Y).
- Кликабельная — открывает соответствующую полную карту.

5. Локальная разработка через Docker

Решение: вся разработка ведётся в Docker. Это решает «у меня работает, у тебя — нет», даёт быстрый онбординг новых разработчиков и упрощает CI.

5.1. Что включено в dev-стек

Сервис	Образ	Порт	Назначение
postgres	postgres:16-alpine	5432	Основная БД
redis	redis:7-alpine	6379	Кэш, очереди, сессии
backend	node:20-alpine + локальный Dockerfile	3000	Fastify API
worker	тот же Dockerfile, другая команда	—	BullMQ воркер для тиков
frontend	node:20-alpine + Vite dev server	5173	Telegram Mini App
adminer	adminer:latest	8080	GUI для Postgres (только dev)
redis-commander	rediscommander/redis-commander	8081	GUI для Redis (только dev)

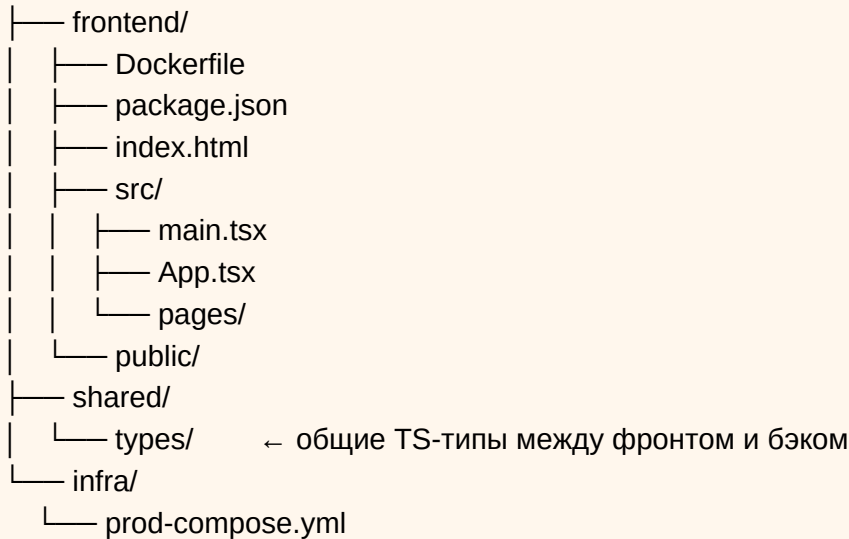
5.2. Структура папок репозитория

Дерево

```

new-universe/
├── docker-compose.yml
├── docker-compose.override.yml.example
├── .env.example
├── README.md
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   ├── src/
│   │   ├── index.ts
│   │   ├── routes/
│   │   ├── services/
│   │   ├── workers/
│   │   └── db/migrations/
│   └── tests/

```



5.3. Команды разработки

- `docker compose up -d` — поднять postgres + redis + backend + frontend.
- `docker compose up -d adminer redis-commander` — UI для БД (опционально).
- `docker compose logs -f backend` — следить за логами бэкенда.
- `docker compose exec backend npm run db:migrate` — накатить миграции.
- `docker compose exec backend npm run db:seed` — засеять справочники (ресурсы, типы зданий).
- `docker compose exec backend npm run test` — прогон тестов.
- `docker compose down -v` — остановить и УДАЛИТЬ volumes (полная очистка).

5.4. Hot reload

- Backend: `src/` монтируется в контейнер, `tsx watch` перезапускает на изменениях.
- Frontend: `src/` монтируется, Vite HMR работает поверх.
- Миграции БД — не автоматически, по команде, чтобы не было сюрпризов.

5.5. Тестирование Telegram Mini App локально

- Использовать `ngrok` или `cloudflared tunnel` чтобы пробросить `localhost:5173` наружу с `https`.
- Установить URL туннеля в `@BotFather` как Web App URL для тестового бота.
- Аналогично — туннель для backend webhook (через `cloudflared`, бесплатно).
- Альтернатива: использовать Telegram Test Servers — отдельная сеть Telegram для тестов, без блокировок.

5.6. Production-ready образы

- Multi-stage Dockerfile: отдельные стадии для `install`, `build`, `run`.

- Финальный образ: distroless или alpine с минимумом пакетов.
- Размер цели: backend ≤ 200 MB, frontend (статика) ≤ 30 MB.
- CI: GitHub Actions собирает образ → пушит в GHCR → деплоит на Hetzner через docker-compose pull.

6. Миграционный план — что делать с уже написанными разделами GDD

Этот документ — additive: ничего из основного GDD не вычёркивается, дополняются конкретные разделы. Когда добавлять/менять.

Раздел в GDD v1.0	Действие	Что добавить
§3 Ресурсы	дополнить	§3.7 → Лёд ⇌ Вода (см. §2 этого аддендума)
§4 Планеты, системы, галактика	частично заменить	§4.1-§4.4 переписать по модели §3 этого аддендума
§5 Здания	дополнить	+ Криогенный завод
§6 Корабли	дополнить	+ Jump Ship класс, + Прыжковое топливо как ресурс
§9 Мультиплеер	уточнить	§9.6: Home System НЕ атакуется, остальные ограничения остаются
§10 Прогрессия	дополнить	+ ветка Jump Drive (5 уровней)
§14 Технический стек	дополнить	+ Docker Compose dev workflow (см. §5)
§16 Roadmap	без изменений	—

6.1. Когда формально обновить основной GDD

- Аддендум v1.1 — это записка перед началом разработки. По мере имплементации могут вылезти новые правки → копируем v1.2.
- Когда количество правок > ~10 крупных, делаем slip-cover GDD v2.0, в котором всё консолидировано в один документ.
- До MVP лучше держать «GDD v1.0 + аддендумы», чтобы видеть что менялось и почему. Это отличный артефакт пост-мортема: «вот что мы поняли в процессе».

7. Сводка изменений (для имплементаторов)

Кратко всё, что должны учесть инженеры при стройке системы.

Чек-лист имплементатора

- ☐ В БД: добавить таблицу discovered_planets (player_id, planet_id, discovered_at) и discovered_systems аналогично.
- ☐ Все API, возвращающие планеты/системы, фильтруются по этой таблице для текущего игрока.
- ☐ Алгоритм генерации home system: детерминированный, фиксируемый по seed = hash(user_id, server_secret).
- ☐ Procedural generation секторов: lazy, по триггерам прыжка/density check.
- ☐ Home System помечается флагом is_home_system = true, сервер блокирует любые «чужие» обращения к ней.
- ☐ Tritium — новый ресурс Tier 3, добавить в seed и в home-системы.
- ☐ Криогенный завод — новый building_type.
- ☐ Jump Ship — новый ship_type, требует Jump Drive tech.
- ☐ Jump Fuel — новый craft_recipe.
- ☐ Sensor visibility tick — новая фоновая задача, проверяющая корабли в полёте.
- ☐ Конвертация Лёд ⇌ Вода — рецепт в craft_recipes.
- ☐ UI: 3 карты (Home, Sector, System) + minimap.
- ☐ Docker compose dev — рабочее с первого «docker compose up».
- ☐ Все имена сервисов, БД, образов используют new-universe / new_universe.